

CPSC 483: Introduction to Machine Learning

Instructor: Anli Ji

Fall 2026

Cal State Fullerton

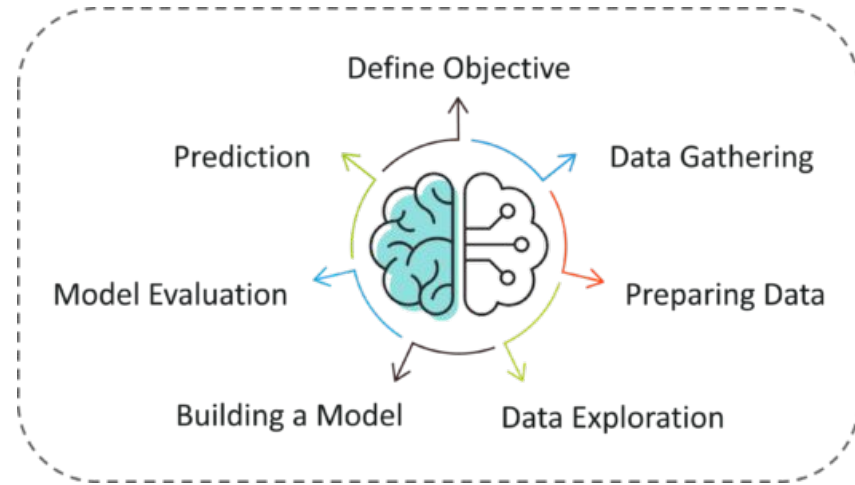
Chapter 2

Data Preprocessing

- Exploratory data analysis
- Test-train split
- Sampling strategies
- Feature transforms
- Feature scaling
- Handling missing data
- Data processing pipelines

Typical Machine Learning Steps

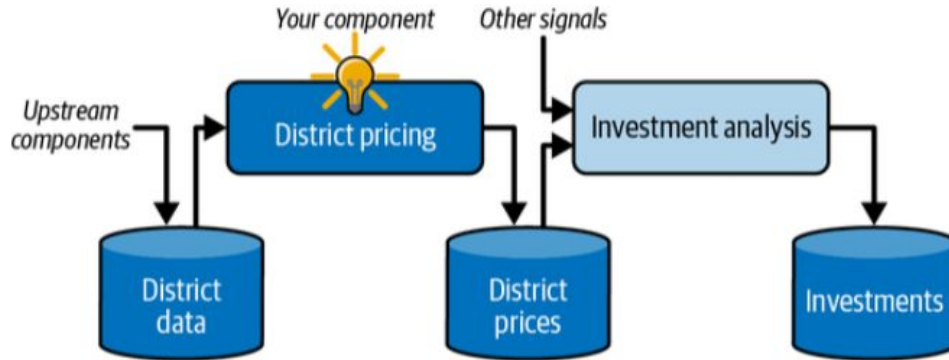
- Look at the big picture
- Get the data
- Explore and visualize the data to gain insights
- Prepare the data for ML algorithms
- Select a model and train it
- Fine-tune your model
- Present your solution
- Launch, monitor, and maintain your system



Look at the Big Picture

Two questions to ask

- What is the overall objective?
 - Building a ML model is NOT the end goal!
 - Determines which performance measure to use to evaluate the model
- What does the current solution look like (if any)?
 - Baseline performance



Recap: Types of ML

Type of output

- Classification
- Regression
- Clustering

Type of training supervision

- Supervised
- Unsupervised
- Semi-supervised
- Self-supervised
- Reinforcement Learning

Get the Data

Popular open data repositories

- [UC Irvine Machine Learning Repository](#)
- [Kaggle datasets](#)
- [Amazon's AWS datasets](#)

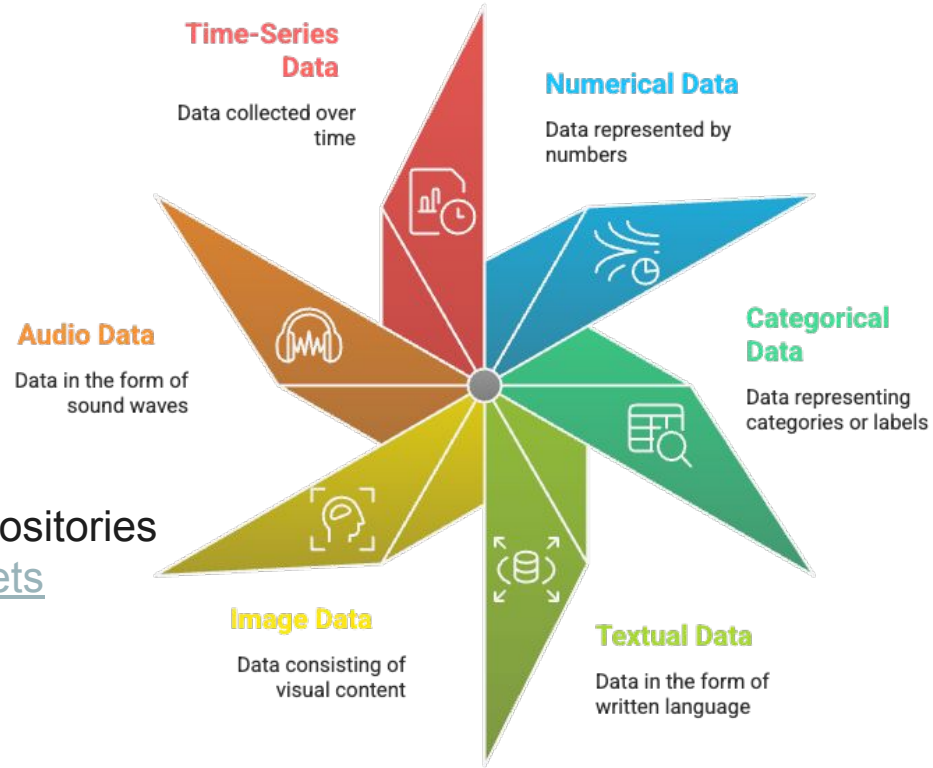
Meta portals (they list open data repositories)

- [Data Portals](#)
- [OpenDataMonitor](#)

Other pages listing many popular open data repositories

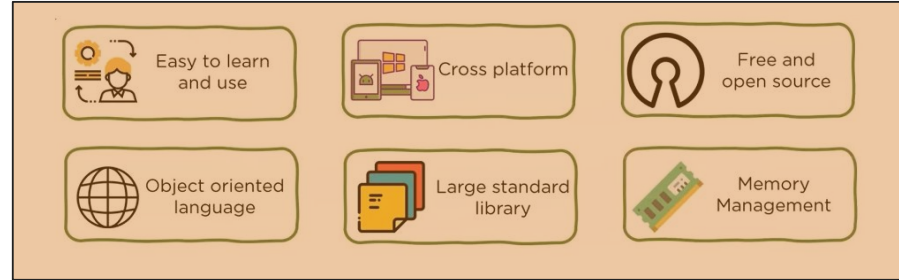
- [Wikipedia's list of Machine Learning datasets](#)
- [Quora.com](#)
- [The datasets subreddit](#)

Overview of Machine Learning Data Types

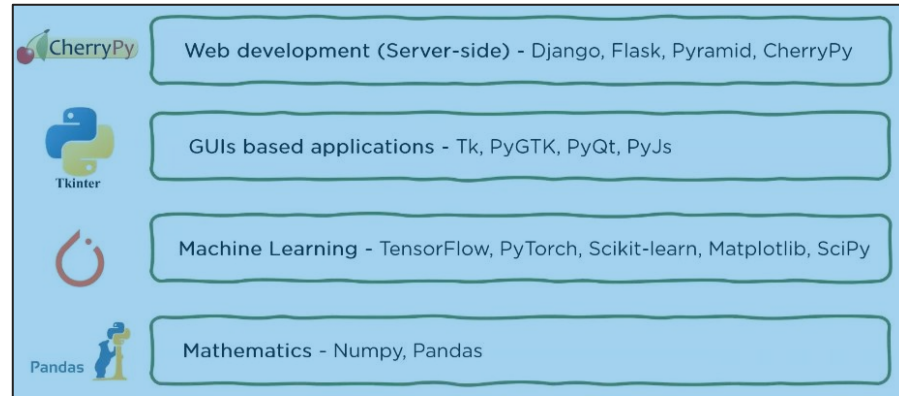


Python for Machine Learning

Features of Python



Python Frameworks/Libraries



- Numpy: Fast numerical data processing and computation (>10x faster than Python)
 - <https://numpy.org>
- Pandas: Data loading, structuring, cleaning, searching, processing
 - <https://pandas.pydata.org>, <https://www.w3schools.com/python/pandas/default.asp>
- Statistics: Statistics libraries
 - <https://docs.python.org/3/library/statistics.html>
 - <https://scipy-lectures.org/packages/statistics/index.html>
- Scipy: Scientific computation
 - https://docs.scipy.org/doc/scipy/getting_started.html
- Matplotlib: Plotting libraries
 - <https://matplotlib.org/stable/tutorials/introductory/pyplot.html>
- Scikit-learn: ML packages using numpy, pandas, scipy, matplotlib without ANN
 - <https://scikit-learn.org>
- PyTorch: ANN libraries including deep learning and natural language processing
 - <https://pytorch.org/tutorials/>
- Keras: Deep learning framework with APIs built on top of TensorFlow
 - <https://keras.io/>
- Anaconda: Python distribution including most packages
 - <https://www.anaconda.com>

Python Packages for Machine Learning

Python IDEs



Jupyter
notebook



Visual Studio
Code



Sublime text



PyCharm



IDLE



Spyder
Notebook

pandas: Data Table Representation

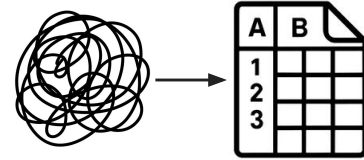


Series & DataFrames

- Series is a column
- DataFrame is a two-dimensional table made up of a collection of Series
- DataFrames and Series are similar in that many operations can be done on both
 - such as filling in null values and calculating the mean

pandas is a fast, powerful, flexible and easy to use open source data analysis and manipulation tool, built on top of the Python programming language.

Series			Series			DataFrame	
apples			oranges			apples	oranges
0	3	+	0	0	=	0	3
1	2		1	3		1	2
2	0		2	7		2	0
3	1		3	2		3	1



pandas.DataFrame

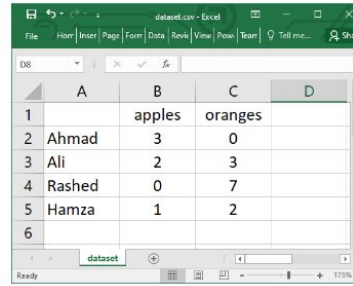
```
pandas.DataFrame(data, index, columns, dtype, copy)
```

- **data:** Can be *ndarray, series, map, lists, dict*, constants, another *DataFrame*.
- **index:** The row labels. Optional, default is *0, 1, 2, ...*
- **columns:** The column labels. Optional, default is *0, 1, 2, ...*
- **dtype:** Data type of each column.
- **copy:** use a copy of data or a reference

pandas: Data Table Representation

A DataFrame can be created from other data structures such as

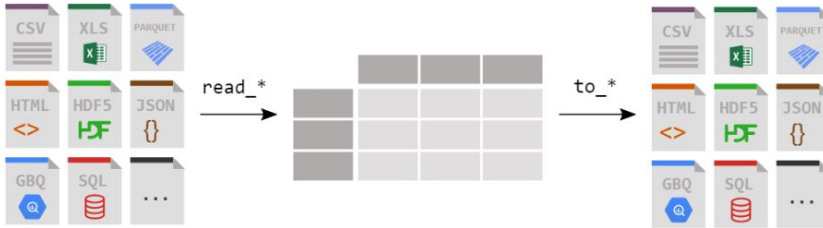
- Lists
- dict
- Series
- Numpy ndarrays
- Another DataFrame



	A	B	C	D
1		apples	oranges	
2	Ahmad	3	0	
3	Ali	2	3	
4	Rashed	0	7	
5	Hamza	1	2	
6				



```
File Edit Format Run Options Window Help
1 import pandas as pd
2
3 df = pd.read_csv('dataset.csv')
4 print(df)
5
6 # OR
7
8 df = pd.read_csv('dataset.csv', index_col=0)
9 print(df)
10
```



What information can we learn?

Common summary statistics

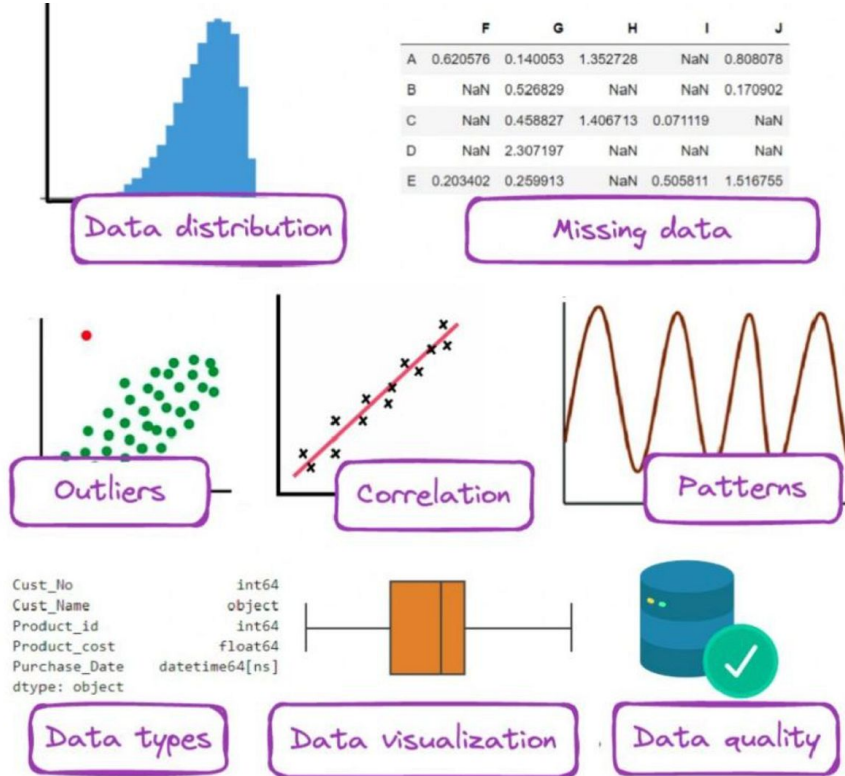
```
df.head()           # First 5 rows
df.tail()           # Last 5 rows
df.info()           # Summary of the DataFrame
df.describe()       # Statistics of numerical columns
df.shape            # Shape of the DataFrame
df.columns          # List of column names
df.dtypes           # Data types of columns
df.sample(5)        # Random 5 rows
```

```
df.mean()           #Compute mean of columns.
df.median()         #Compute median of columns.
df.sum()            #Compute column-wise or row-wise sum.
df.min()            #Minimum values in columns.
df.max()            #Maximum values in columns.
df.count()          #Count non-missing values in each column.
df.groupby('column') #Group data by a column.
df.agg({'col1': 'mean', 'col2': 'sum'}) #Apply multiple aggregation function
s.
df['column'].mean()   # Mean of a column
df.groupby('group').sum() # Group by and aggregate
df.agg({'col1': 'mean', 'col2': 'sum'}) # Multiple aggregations
```

A good *pandas* usage cheat sheet

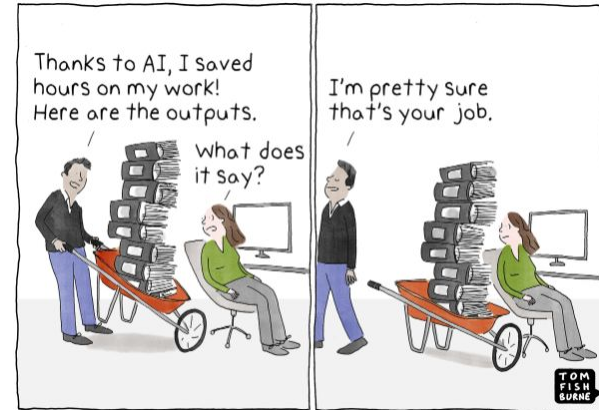
https://pandas.pydata.org/Pandas_Cheat_Sheet.pdf

Exploratory Data Analysis (EDA) - *What it is?*



First step to building a model

- Data-driven (model-free)
- Goal: get a general sense of the data
 - Getting your hands “dirty” with the data
- Interactive and visual
 - Humans are good at pattern recognition



© marketoonist.com

Exploratory Data Analysis (EDA) - Types

- Univariate: studies **one variable** at a time
- Bivariate: examines the relationship between **two variables**
- Multivariate: studies **three or more variables** together



seaborn

<https://seaborn.pydata.org/>

UNIVARIATE



One variable

BIVARIATE

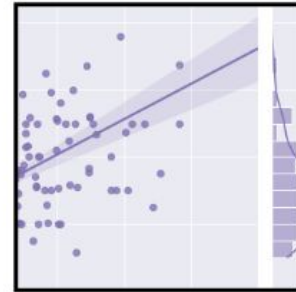
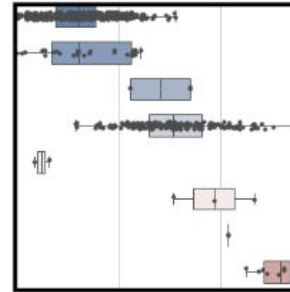
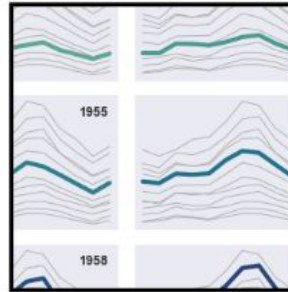
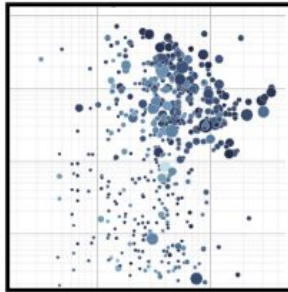
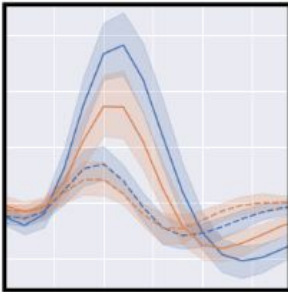
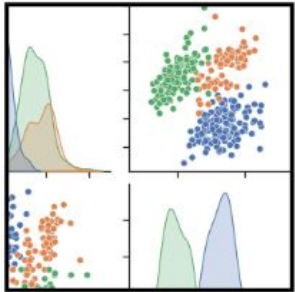


Two variables

MULTIVARIATE



Multiple variables



How to Check Data Quality?

Data Cleaning

Missing values? Outliers?

- Count the NA/Nulls
- Possible that missing values are not stored as NA
 - Instead, 0 or -1 or something that cannot be a valid value

Common Handling Strategies:

- Drop NA (either remove rows or columns)
- Imputation/Fill in (i.e., mean or median)

df.dropna()

Drop rows with any column having NA/null data.

df.fillna(value)

Replace all NA/null data with value.

df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 8 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Transaction ID   10000 non-null  object
1   Item             9667 non-null   object
2   Quantity         9862 non-null   object
3   Price Per Unit   9821 non-null   object
4   Total Spent      9827 non-null   object
5   Payment Method   7421 non-null   object
6   Location         6735 non-null   object
7   Transaction Date 9841 non-null   object
dtypes: object(8)
memory usage: 625.1+ KB
```

	Transaction ID	Item	Quantity	Price Per Unit	Total Spent	Payment Method	Location	Transaction Date
0	TXN_1961373	Coffee	2	2.0	4.0	Credit Card	Takeaway	2023-09-08
1	TXN_4977031	Cake	4	3.0	12.0	Cash	In-store	2023-05-16
2	TXN_4271903	Cookie	4	1.0	ERROR	Credit Card	In-store	2023-07-19
3	TXN_7034554	Salad	2	5.0	10.0	UNKNOWN	UNKNOWN	2023-04-27
4	TXN_3160411	Coffee	2	2.0	4.0	Digital Wallet	In-store	2023-06-11
...	...	...	...	...	...	...	...	...
9995	TXN_7672686	Coffee	2	2.0	4.0	NaN	UNKNOWN	2023-08-30
9996	TXN_9659401	NaN	3	NaN	3.0	Digital Wallet	NaN	2023-06-02

Numerical vs. Categorical Variables

Numerical variables

- Values can be ordered, i.e., there is a natural ordering between the values
 - i.e., Continuous/Discrete
- E.g., *Age* (younger/old), *Weight* (lighter/heavier)
 - Can calculate averages

Categorical variables

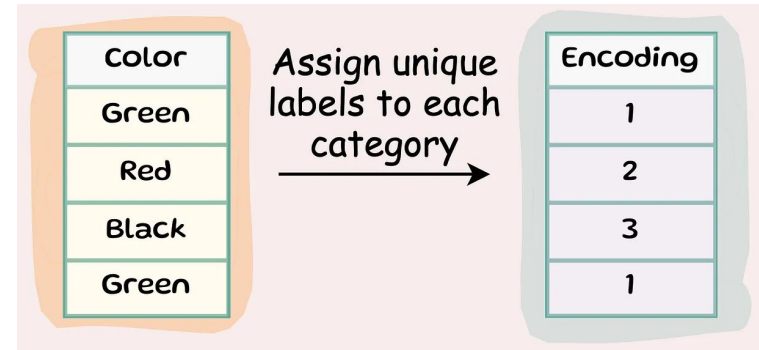
- There is no natural ordering between the values
 - i.e., Nominal/Ordinal
- E.g., *Marital status*: Single, Married, Divorced
- E.g., *Color*: red, blue, green, ...
 - Can NOT compute averages

Converting Categorical features to Numerical

Ordinal/Label Encoding

- Assign an integer/label to every unique categorical value
 - Color = [Green, Red, Black, Green, ...]
 - Color_encoded = [1, 2, 3, 1, ...]

```
from sklearn.preprocessing import OrdinalEncoder  
ordinal_encoder = OrdinalEncoder()  
Color_encoded = ordinal_encoder.fit_transform(Color)
```



Reference:

<https://blog.dailydoseofds.com/p/7-categorical-data-encoding-techniques>

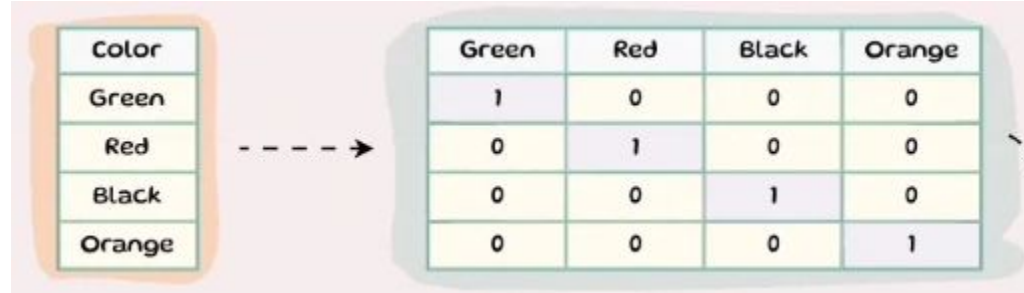
Converting Categorical features to Numerical

One-hot Encoding

- Create new columns, one for *each unique value*
- Assign 0/1 to each column depending on the value

Sparse matrix

- Color = [Red, Blue, Green, Blue, ...]
- Color_Red = [1, 0, 0, 0, ...]
- Color_Blue = [0, 1, 0, 1, ...]
- Color_Green = [0, 0, 1, 0, ...]



```
from sklearn.preprocessing import OneHotEncoder  
cat_encoder = OneHotEncoder()  
Color_onehot = cat_encoder.fit_transform(Color)
```

Reference:

<https://blog.dailydoseofds.com/p/7-categorical-data-encoding-techniques>

Practice!!!

- Given the categorical variable of “Standing” in the table
- Perform:
 - Ordinal Encoding
 - One-hot Encoding
- Do this by hand
- Do this with code
 - First create the *DataFrame*

CWID	Standing	GPA
5460	Freshman	2.8
2950	Junior	3.2
4544	Freshman	2.7
8902	Senior	3.3